

Plausible Fiction on MeTTaIL

A Requirements & Design Document for a Typed Marketplace of Programmatic Intentions

F1R3FLY.io · in dialogue with D. Spivak (Topos Institute)

June 28, 2026 — Draft for discussion

Abstract

Plausible Fiction (PF) is a marketplace in which a participant publishes a *programmatic* description of a good or service they want produced — “plausible” because it is a runnable artifact, “fiction” because it contains *holes* that no one party can yet fill. Following Spivak’s original framing, a PF is a term in a dependently typed lambda calculus; a PF with holes is a *context*. We refine that framing by distinguishing holes that must be filled by *other agents* in the market from holes that are filled by *ordinary computation*. This document specifies the system in two layers on top of MeTTaIL (rholang 1.4): (i) an object language — a dependently typed λ -calculus with typed holes — given as a MeTTaIL **language!** definition, whose terms ride on rho channels; and (ii) a coordination protocol written in MeTTaIL’s built-in embedded rholang that manages the full PF lifecycle: publish, match, broker, ground, run, monitor, and either promote to a *template* on success or *retire* (hide or advertise) on failure. We give code illustrations throughout, a lifecycle state machine, a numbered requirements catalogue, a worked end-to-end example, and an honest accounting of what is established versus conjectural.

Contents

1	Purpose and scope	2
2	Conceptual model	2
2.1	A PF is a typed context	2
2.2	Grounding and running	3
2.3	Two kinds of hole	3
2.4	The market is the search procedure	3
3	Architecture	3
4	Object language PFLam	4
4.1	Hole interfaces are telescopes	4
4.2	The language! specification	4
4.3	Bidirectional typing and the matching relation	6
5	The bridge	6
6	Coordination protocol (embedded rholang)	7
6.1	Namespaces and channel conventions	7
6.2	Publish: Draft $\rightarrow$ Published	7
6.3	Offer: supply side	8
6.4	Broker: forwarders with optional type filtering	8
6.5	Partial grounding and recursion	9

6.6	Grounding detection: $\rightarrow$ Grounded	9
6.7	Execution: Grounded $\rightarrow$ Running	9
6.8	Monitoring, barriers, and coordination	9
6.9	Success $\rightarrow$ Template	10
6.10	Failure $\rightarrow$ Retirement: hide vs. advertise	10
7	Lifecycle state machine	11
8	Requirements catalogue	12
9	Worked example	13
10	Established vs. conjectural; open problems	14
	Glossary	15

1 Purpose and scope

This document specifies *what* the Plausible Fiction service must do and *how* it maps onto MeTTaIL, at a level of detail sufficient to begin implementation and to anchor continued design discussion with the Topos Institute. It is deliberately concrete about the MeTTaIL surface: the object language is given as a **language!** specification, and every coordination behaviour is given as embedded rholang. It is deliberately honest about its open problems (§10): several matching and templating questions are undecidable in full generality, and the design’s central move is to *delegate* those problems to the market rather than to a solver.

Non-goals. We do not re-specify rholang itself. MeTTaIL ships an embedded rholang as its coordination language, and separately offers a language-definition facility (it can even give a meta-circular account of rho); the latter is what we use to define the object language, and the former is what we use to coordinate. We treat both as given.

2 Conceptual model

2.1 A PF is a typed context

Fix a dependently typed λ -calculus with a sort **Term** of terms (which, being dependent, also serves as the sort of types). A *Plausible Fiction* is a **Term** that may contain *typed holes*. Writing $?h:T$ for a hole named h with expected type T :

$$\text{PF} ::= t \quad \text{where } t \text{ ranges over Term with holes } ?h:T.$$

A hole is a typed metavariable; a term with holes is exactly a *many-holed context*. The two defining slogans become precise:

- **Plausible** = the description is a real, type-checkable program fragment, not prose.
- **Fiction** = it has holes; it asserts the *existence* of fillers it does not yet possess.

2.2 Grounding and running

A PF is *grounded* when it has no *agent* holes (defined next). A grounded PF is a closed program that can be *run*: every party who contributed a sub-PF performs their part, in the world, under coordination and monitoring. “Filling a hole” is plugging another PF into it; when the last agent hole is plugged the PF becomes grounded and is eligible to run.

2.3 Two kinds of hole

This is our refinement of Spivak’s model, and it drives the whole lifecycle.

Agent holes $?h:T$.

Filled by *another PF supplied by another participant*. These are market demands. They are the only holes that the lifecycle advertises, brokers, and waits on. grounded counts *only* agent holes.

Computation holes.

Filled by *standard computation*: β/η -reduction, normalization, and elaboration of implicit arguments (ordinary metavariables solved by unification). These never become market demands; they are discharged by the evaluator/elaborator.

Concretely: a redex such as $(\lambda(x:A).b)a$ is a computation hole — it is “filled” by β . An implicit argument that the elaborator can infer is a computation hole. A slot for “a licensed electrician who will wire the venue” is an agent hole — no amount of computation produces the electrician; *another participant* must offer one.

2.4 The market is the search procedure

A subtle but load-bearing observation. “Types mediate matching” means a candidate filler q fits a hole $?h:T$ iff q *checks* at type T in the hole’s local context. *Checking a given q against T is decidable* (bidirectional type-checking, assuming normalization). *Finding* such a q is higher-order search and is undecidable in general. The PF design’s answer is not to run a solver: it is to publish the demand and let other agents *offer* candidates, which the type guard merely *checks*. The rho market — forwarders, brokers, offers — is the (distributed, human-and-machine, economically incentivized) search procedure; the type system is only the acceptance test. See §10 for the formal consequences.

3 Architecture

Three strata (Figure 1):

1. **Object language PFLam** (§4): a dependently typed λ -calculus with typed holes, defined with MeTTail’s **language!**. Pure, total operations: bidirectional typing, holes($\cdot$), grounded($\cdot$), and $\triangleleft$.
2. **Bridge** (§5): a small set of rho-callable predicates/operators — $\text{pf} = \text{ty}|$, $\text{grounded}(\text{pf})$, $\text{plug}(\text{pf}, h, q)$, $\text{holes}(\text{pf})$ — that lift PFLam operations into embedded rholang as Bool/payload folds. This is the only place the two layers touch.
3. **Coordination** (§6): embedded rholang protocols implementing the lifecycle over per-user channel namespaces.

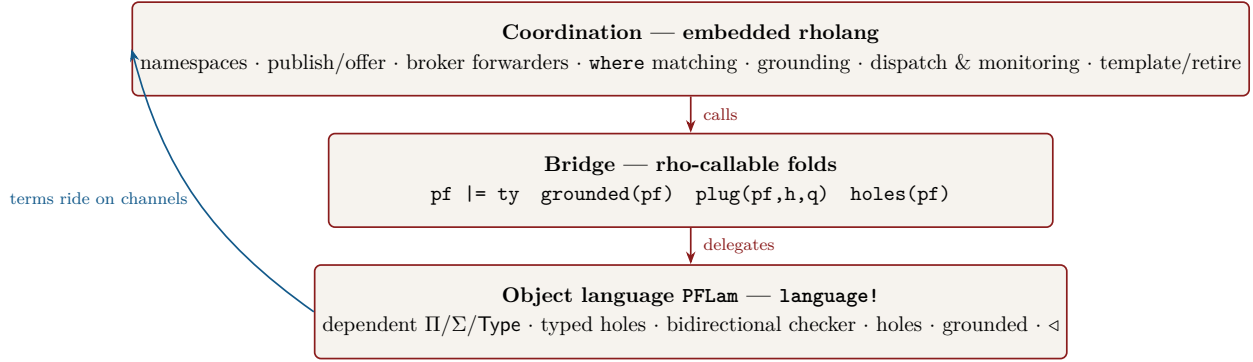


Figure 1: The three strata. Object-language terms are payloads carried on rho channels; the bridge is the sole coupling point.

4 Object language PFLam

We give PFLam as a Pure-Type-System–style dependent calculus with a single sort `Term`, a cumulative universe hierarchy, dependent function (Π) and pair (Σ) types (records-as- Σ model goods/services), and the distinguished HOLE constructor. The surface syntax is ASCII to match MetTail’s existing examples (`lam x.x`, `(f, x)`).

4.1 Hole interfaces are telescopes

A hole does not float free: it sits under binders and may use the variables in scope at its position. Its *interface* is therefore a *telescope* Γ_h (the local context) together with an expected type T_h . We encode this by Π -closing: a hole records the single closed type $\Pi\Gamma_h.T_h$, and a filler is a closed term of that type, *applied* to the local variables at the plug site. This is exactly why Spivak’s hole-filling reads as application: plugging `alicePF` at hole h with q is, up to bookkeeping, `alicePF` with q substituted for $?h:\Pi\Gamma_h.T_h$ and applied to Γ_h . The minimal spec below keeps HOLE binary (h, T) with the telescope folded into T via Π .

4.2 The language! specification

```

language! {
  name: PFLam,

  types {
    Term                                // dependent: terms and types share one sort
    ![String] as HoleId                 // agent-hole identifiers (market-visible)
    ![String] as MetaId                 // computation-hole / elaboration metavaris
    ![u64]   as Lvl                     // universe levels
  },

  terms {
    // ---- core dependent calculus -----
    Univ . l:Lvl                        |- "Type" l                : Term;
    Var . x:Term                         |- x                        : Term; // bound via HOAS below

    Pi . dom:Term, ^x.cod:[Term → Term]
      |- "Pi" "(" x ":" dom ")" "." cod : Term;

    Lam . dom:Term, ^x.body:[Term → Term]
  }
}

```

```

    |- "lam" "(" x ":" dom ")" "." body : Term;
App . fun:Term, arg:Term      |- "(" fun " " arg ")" : Term;

// dependent records model goods/services as Sigma-bundles
Sig . fst:Term, ^x.snd:[Term → Term]
    |- "Sig" "(" x ":" fst ")" "." snd : Term;
Pair . a:Term, b:Term        |- "<" a ", " b ">" : Term;
Fst . p:Term                 |- "fst" p : Term;
Snd . p:Term                 |- "snd" p : Term;

// ---- THE distinguishing constructors -----
// Agent hole: filled by another PF from another participant.
// ty is the Pi-closed interface Pi Gamma_h . T_h (see sec. 4.1).
Hole . h:HoleId, ty:Term      |- "?" h ":" ty : Term;

// Computation hole: an elaboration metavariable, solved by unification.
// NEVER advertised to the market; discharged by the elaborator.
Meta . m:MetaId, ty:Term      |- "%" m ":" ty : Term;

// ---- pure operations exposed to the bridge -----
// grounded(p): no AGENT holes remain (Meta holes are allowed).
Grounded . p:Term |- "grounded" "(" p ")" : Term ![{
  crate::pflam::no_agent_holes(&p) // → Term::BoolLit(_)
}] fold;

// holes(p): the open agent holes as a list of <h, ty> pairs.
HolesOf . p:Term |- "holes" "(" p ")" : Term ![{
  crate::pflam::agent_holes(&p) // → list of pairs
}] fold;

// plug(p, h, q): fill agent hole h in p with filler q (capture-avoiding;
// q is the Pi-closed filler, re-applied to the local telescope).
Plug . p:Term, h:HoleId, q:Term
    |- "plug" "(" p ", " h ", " q ")" : Term ![{
  crate::pflam::plug_hole(&p, &h, &q)
}] fold;
},

equations {
  // beta / eta / projection are CONVERSION (the equational theory used by
  // type conversion and by the matching test |=).
  Beta . |- ( (lam (x : A) . b) a ) = (subst b x a) ;
  Eta . |- (lam (x : A) . (f x)) = f ; // x not free in f
  SigB1 . |- (fst < a , b >) = a ;
  SigB2 . |- (snd < a , b >) = b ;
},

rewrites {
  // directed reduction = "filling computation holes by standard computation"
  BetaRw . |- ( (lam (x : A) . b) a ) ~> (subst b x a) ;
  Fst1 . |- (fst < a , b >) ~> a ;
  Snd1 . |- (snd < a , b >) ~> b ;
  // plug reduces structurally once its filler is present
  PlugRw . |- (plug p h q) ~> (subst_hole p h q) ;
  // congruences (elided): AppCongL/R, PiCong, LamCong, SigCong, PairCong, ...
},

logic {
  // Bidirectional typing as an ascent relation (sketch, sec. 4.3).

```

```

// check(ctx, term, ty) and synth(ctx, term, ty).
// matching test: models(q, ty) iff exists A. synth([], q, A), sub(A, ty).
relation check(Ctx, Term, Term);
relation synth(Ctx, Term, Term);
relation sub(Term, Term); // conversion + universe cumulativity
relation models(Term, Term);
models(q, ty) <-- synth(ctx0(), q, a), sub(a, ty);
},
}

```

4.3 Bidirectional typing and the matching relation

The equational theory ($\beta\eta$, projections) is *conversion*, written $\equiv$. Subtyping $<:$ is conversion plus universe cumulativity (and, optionally, width/depth subtyping on Σ -records, which is exactly what lets a richer offer satisfy a leaner demand). The judgments are the usual bidirectional pair; the rules that matter for matching are:

$$\begin{array}{c}
\text{HOLE-SYNTH} \\
\hline
\Gamma \vdash ?h : T \Rightarrow T
\end{array}
\quad
\begin{array}{c}
\text{SUB} \\
\hline
\frac{\Gamma \vdash t \Rightarrow A \quad A <: B}{\Gamma \vdash t \Leftarrow B}
\end{array}
\quad
\begin{array}{c}
\text{PI} \\
\hline
\frac{\Gamma \vdash A \Leftarrow \text{Type}_i \quad \Gamma, x:A \vdash B \Leftarrow \text{Type}_i}{\Gamma \vdash \Pi(x:A).B \Rightarrow \text{Type}_i}
\end{array}$$

$$\begin{array}{c}
\text{LAM} \\
\hline
\frac{\Gamma, x:A \vdash b \Leftarrow B}{\Gamma \vdash \lambda(x:A).b \Leftarrow \Pi(x:A).B}
\end{array}
\quad
\begin{array}{c}
\text{APP} \\
\hline
\frac{\Gamma \vdash f \Rightarrow \Pi(x:A).B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash f a \Rightarrow B[a/x]}
\end{array}$$

Matching. A candidate filler q *satisfies* a hole $?h : T$ — written $q \models T$ — iff $\Gamma_h \vdash q \Leftarrow T$, equivalently (SUB) iff q synthesizes some A with $A <: T$. This is the predicate the rho **where** guard evaluates. It is decidable on the chosen normalizing fragment; the search for q is *not* performed here (§2.4).

Plugging, precisely. For $q \models T_h$,

$$p \triangleleft_h q := \text{subst_hole}(p, h, q),$$

the capture-avoiding replacement of every $?h : T_h$ in p by q (with q applied to the local telescope Γ_h per §4.1). *Crucially, q may itself contain agent holes.* Then

$$\text{holes}(p \triangleleft_h q) = (\text{holes}(p) \setminus \{h\}) \cup \text{holes}(q),$$

which is the recursive engine of the marketplace: filling a hole can introduce new holes, deferred to new participants. $\text{grounded}(p)$ holds iff $\text{holes}(p) = \emptyset$.

5 The bridge

MeTTAIL lets a defined language’s terms be sent on rho channels. The coordination layer therefore carries PFLam terms as payloads and needs four host predicates/operators, each a rho fold that delegates to PFLam. We extend the embedded coordination language with:

```

// Bridge operators (rho-level folds delegating to PFLam). Surface syntax in
// the coordination layer; pf, ty, q are embedded PFLam terms carried as payload.

```

```

Models . pf:Proc, ty:Proc |- pf "=" ty : Proc ![{
  // decidable bidirectional check: PFLam.check(ctx_h, pf, ty)
  bool_proc(crate::pflam::models(&pf, &ty))
}] fold;

GroundedP. pf:Proc |- "grounded" "(" pf ")" : Proc ![{
  bool_proc(crate::pflam::no_agent_holes(&pf))
}] fold;

PlugP . pf:Proc, h:Proc, q:Proc
  |- "plug" "(" pf ", " h ", " q ")" : Proc ![{
    crate::pflam::plug_hole_payload(&pf, &h, &q) // → embedded PFLam term
  }] fold;

HolesP . pf:Proc |- "holes" "(" pf ")" : Proc ![{
  crate::pflam::agent_holes_payload(&pf) // → List of [h, ty]
}] fold;

```

With these in scope, the matching guard Spivak’s framing asks for is literally a rholang **where** clause: **where** pf= ty|. Everything in §6 is ordinary embedded rholang plus these four operators.

6 Coordination protocol (embedded rholang)

6.1 Namespaces and channel conventions

To each user (say *alice*) we associate a namespace of channels, addressed by *quoting* structured names. We write channels as quoted lists; e.g. @[alice, "demand", h] is the demand channel for hole *h* of one of Alice’s PFs. Conventions:

Channel	Role
@[u, "pf", pfId]	canonical store of user <i>u</i> ’s PF <i>pfId</i> (latest term)
@[u, "demand", pfId, h]	open agent hole <i>h</i> : candidates are offered here
@[u, "supply"]	<i>u</i> publishes grounded/partial PFs it offers to others
@[u, "prov", pfId]	provenance map: hole $\mapsto$ (party, fillerId)
@[u, "ground", pfId]	fires once <i>pfId</i> becomes grounded
@[u, "run", pfId]	execution controller for a grounded <i>pfId</i>
@[u, "step", pfId, k]	attestation channel for execution step <i>k</i>
@["mkt", "templates"]	public registry of successful PFs (templates)
@["mkt", "caveats"]	public registry of advertised failures (anti-templates)

6.2 Publish: Draft → Published

Publishing stores the PF and emits one persistent *demand* per open agent hole. Each demand is a type-guarded receiver: it accepts only candidates that satisfy the hole’s type, and on acceptance it plugs and re-publishes. This is the generalization of Spivak’s **for**(pf <- aliceChan **where** pf= type) aliceChan!((alicePF pf)) |.

```

// publish(alice, pfId, alicePF): store, then arm one demand per agent hole.
new publish in {
  contract publish(@u, @pfId, @pf) = {

```

```

@[u, "pf", pfId]!!( pf )           // canonical store (persistent)
| for( @hs <- holes(pf) ) {         // hs : List of [h, ty]
    // arm a guarded demand for every open agent hole
    match hs {
        [] => @[u, "ground", pfId]!( pf )    // already grounded
        [[h, ty] ...] => armDemand!( u, pfId, h, ty )
    }
}
|
// one guarded, persistent demand for hole h with expected type ty
contract armDemand(@u, @pfId, @h, @ty) = {
    for( @cand <= @[u, "demand", pfId, h] where cand != ty ) { // type mediates matching
        for( @pf <- @[u, "pf", pfId] ) { // read current term
            new pf2 in {
                pf2!( plug(pf, h, cand) ) | // fill the hole
                for( @next <- pf2 ) {
                    recordProvenance!( u, pfId, h, cand ) |
                    republish!( u, pfId, next ) // partial grounding → recurse
                }
            }
        }
    }
}
}
}
}

```

Notes. (i) The guard `where cand = ty` is the decidable acceptance test of §4.3; unmatched candidates simply do not fire the receive (they remain available for other demands). (ii) The receive is *persistent* (`<=`) so a demand keeps accepting until the hole is filled and the demand is withdrawn (§6.10). (iii) `republish` re-enters *publish*, re-deriving holes of the new term — this is where *q*’s own holes (if any) become fresh demands.

6.3 Offer: supply side

Bob offers a PF by sending it where a broker (or Alice directly) can pick it up. In the minimal form this is Spivak’s `bobChan!( bobPF )`:

```

// Bob advertises a (possibly still-holed) PF he is willing to provide.
@[bob, "supply"]!!( bobPF )

```

6.4 Broker: forwarders with optional type filtering

A broker connects supply to demand by creating forwarders. The bare forwarder is Spivak’s `aliceChan!( pf ) for( pf <- bobChan )`; we add the type guard so the broker only forwards plausibly-matching candidates and does not spam Alice’s demand:

```

// broker(bob → alice@(pfId,h:ty)): forward Bob's supply to Alice's demand,
// pre-filtered by the hole type so only viable candidates flow.
contract broker(@bob, @alice, @pfId, @h, @ty) = {
    for( @cand <= @[bob, "supply"] where cand != ty ) {
        @[alice, "demand", pfId, h]!( cand ) // Alice's guarded demand re-checks and plugs
    }
}

```

The double check (broker filters, demand re-checks) is intentional: the broker’s filter is an optimization; the demand’s guard is the authority. Brokers may be matchmakers, reputation services, or

auctions; none of them can cause an ill-typed plug, because the plug only happens behind Alice's guard.

6.5 Partial grounding and recursion

Each successful plug strictly transforms the hole set per §4.3. `republish` re-arms demands for whatever holes remain (old minus h , plus q 's new holes). The market thus explores a tree of sub-PFs of unbounded depth, each node owned by whichever participant supplied it. Termination of *a particular* PF is exactly: every branch eventually reaches grounded fillers — a liveness property of the market, not a theorem (§10).

6.6 Grounding detection: $\rightarrow$ Grounded

When `holes(pf)` returns [], `publish` emits on `@[u, "ground", pfId]`. That event is the precondition for execution.

6.7 Execution: Grounded $\rightarrow$ Running

Running a grounded PF means: walk its plan, dispatch each leaf obligation to the party who supplied it (read from provenance), and collect attestations. Because the obligations are discharged *in the world*, attestations are signed acknowledgements on per-step channels (oracles); rho coordinates and records, it does not perform the physical work.

```
// run(u, pfId): dispatch obligations and start the monitor.
contract run(@u, @pfId) = {
  for( @pf <- @[u, "ground", pfId] ; @prov <- @[u, "prov", pfId] ) {
    new steps in {
      steps!( plan(pf, prov) ) | // ordered/partially-ordered obligations
      for( @plan <- steps ) {
        dispatch!( u, pfId, plan ) | // notify each party of its step
        monitor!( u, pfId, plan, 0 ) // begin monitoring at step 0
      }
    }
  }
}

// dispatch: tell each contributing party what to do, on their own channel.
contract dispatch(@u, @pfId, @plan) = {
  for( @[party, k, obligation] <= @plan ) { // stream the plan's leaves
    @[party, "todo", pfId, k]!( obligation ) // party performs, then attests on step k
  }
}
```

6.8 Monitoring, barriers, and coordination

The monitor advances only on positive attestation, joins steps that must complete together (a *barrier* via multi-receive `&`), and routes timeouts/negative acks to failure.

```
// monitor advances step-by-step; a barrier step waits on several attestations at once.
contract monitor(@u, @pfId, @plan, @k) = {
  for( @stepK <- stepAt(plan, k) ) {
    match stepK {
      // ordinary step: one attestation
      [party, k, _] =>
```

```

    for( @ack <- @[u, "step", pfId, k] where ack == "ok" ) {
      advance!( u, pfId, plan, k )
    }
    // barrier step: all listed parties must attest before advancing
    [barrier, ks] =>
      for( @a <- @[u, "step", pfId, nth(ks,0)] where a == "ok"
        & @b <- @[u, "step", pfId, nth(ks,1)] where b == "ok" ) {      // ... & ... per ks
        advance!( u, pfId, plan, k )
      }
    }
  }
  // failure routing (timeout or negative attestation) handled in parallel:
  | for( @ack <- @[u, "step", pfId, k] where ack != "ok" ) {
    fail!( u, pfId, pfId )      // → retirement (sec. 6.11)
  }
}

// advance: last step → success; otherwise continue.
contract advance(@u, @pfId, @plan, @k) = {
  match isLast(plan, k) {
    true  => succeed!( u, pfId )
    false => monitor!( u, pfId, plan, k + 1 )
  }
}

```

On timeouts. Pure rho has no clock; “timeout” is supplied by an external timer oracle that posts a non-“ok” attestation on the step channel after a deadline. The monitor treats it uniformly as a failed step.

6.9 Success → Template

If every step attests “ok”, the PF *and its sub-PFs* become reusable templates. Templating is *re-abstraction*: take the grounded composite and reintroduce holes at the positions that were filled (per provenance), yielding a parametric PF that can be re-grounded with different fillers later. The principled version of “reintroduce holes at the filled positions” is anti-unification (least general generalization) over the filled subterms (§10).

```

contract succeed(@u, @pfId) = {
  for( @pf <- @[u, "pf", pfId] ; @prov <- @[u, "prov", pfId] ) {
    new tmpl in {
      tmpl!( abstract(pf, prov) ) |      // re-introduce holes at filled positions
      for( @t <- tmpl ) {
        @["mkt", "templates"]!!( [u, pfId, t] ) // publish reusable template
        | @[u, "supply"]!!( t )           // u can now offer the pattern to others
      }
    }
  }
}

```

6.10 Failure → Retirement: hide vs. advertise

Two retirements, per the brief. Both first *withdraw* the PF’s live demands (consume the persistent demand receivers so no further candidates are accepted). They differ in what they make public.

```

contract fail(@u, @pfId, @reason) = {
  withdrawDemands!( u, pfId ) |                               // stop all guarded demands for this PF
  // caller chooses one of the two retirements:
  Nil
}

// (a) HIDE: keep privately, advertise nothing. Quiet failure.
contract hide(@u, @pfId) = {
  for( @pf <- @[u, "pf", pfId] ) {
    @[u, "hidden", pfId]!!( pf )                               // retained privately; absent from market
  }
}

// (b) ADVERTISE: broadcast the failed pattern so the market routes around it.
//      An anti-template / caveat that other agents can match against.
contract advertise(@u, @pfId, @reason) = {
  for( @pf <- @[u, "pf", pfId] ) {
    @["mkt", "caveats"]!!( [u, pfId, abstract(pf, reason)] ) // public anti-template
  }
}

```

Symmetry worth noting: success publishes to @["mkt", "templates"] (“do this”); loud failure publishes to @["mkt", "caveats"] (“this didn’t work”). Both feed future matching — templates as supply, caveats as filters.

7 Lifecycle state machine

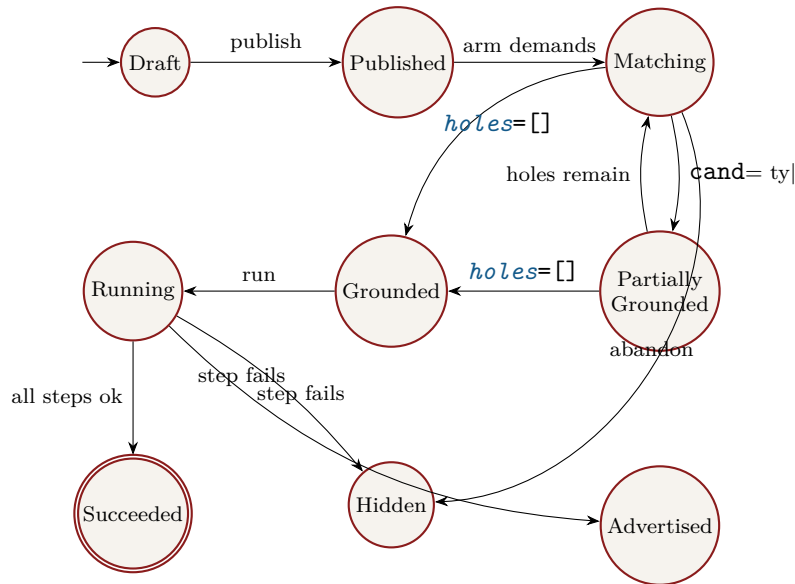


Figure 2: PF lifecycle. *Matching* ↔ *Partially Grounded* is the recursive core; *Succeeded* promotes to a template; failure retires to *Hidden* (quiet) or *Advertised* (loud).

From	To	Guard / event	Coordination effect
Draft	Published	<i>publish</i>	store PF; derive holes
Published	Matching	holes $\neq \emptyset$	arm one guarded demand per hole
Matching	Part. Grounded	<i>cand</i> = ty	<i>plug</i> ; record provenance; republish
Part. Grounded	Matching	holes $\neq \emptyset$	re-arm demands (incl. filler's holes)
Matching / Part.	Grounded	<i>holes</i> (pf) = []	emit @[u, "ground", pfId]
Grounded	Running	<i>run</i>	plan; dispatch obligations; start monitor
Running	Succeeded	all steps "ok"	<i>abstract</i> ; publish template
Running	Hidden	step fails (quiet)	withdraw demands; retain privately
Running	Advertised	step fails (loud)	withdraw demands; publish caveat
Matching	Hidden	abandon	withdraw demands

8 Requirements catalogue

Functional

- FR-1** A PF is a PFLam term with typed agent holes; the system stores each PF on a canonical per-user channel and treats that channel as the single source of truth for its current term.
- FR-2** The system shall distinguish agent holes from computation holes; only agent holes generate market demands, and grounded shall count only agent holes.
- FR-3** For each open agent hole the system shall arm a persistent, type-guarded demand that accepts a candidate iff *cand* = ty|.
- FR-4** Acceptance shall *plug* the candidate, record provenance (hole $\mapsto$ party, filler), and republish the resulting term, re-deriving demands for all remaining and newly introduced holes.
- FR-5** Participants shall be able to offer PFs (supply) and brokers shall be able to create type-filtered forwarders from supply to demand; brokers shall not be able to cause an ill-typed plug.
- FR-6** When a PF's agent-hole set becomes empty the system shall mark it Grounded and make it eligible to run.
- FR-7** Running shall dispatch each leaf obligation to the supplying party (from provenance) and begin monitoring.
- FR-8** Monitoring shall advance only on positive attestation, shall support barrier steps via atomic multi-receive, and shall route negative/late attestations to failure.
- FR-9** On full success the system shall re-abstract the composite into a template and publish it to the public template registry and the owner's supply channel.
- FR-10** On failure the system shall withdraw the PF's live demands and perform exactly one of two retirements: *hide* (retain privately, publish nothing) or *advertise* (publish an anti-template/caveat to the public registry).
- FR-11** Templates shall be re-groundable: a published template is itself a PF with holes and re-enters the lifecycle when adopted.

Non-functional

- NFR-1** *Decidable acceptance.* The matching guard $=|$ must terminate on the supported PFLam fragment (normalizing; conversion decidable).
- NFR-2** *No solver in the loop.* The system must not attempt to synthesize fillers; it only checks offered candidates (§2.4).
- NFR-3** *Capability discipline.* Channel names are unforgeable quoted names; a party can act on a hole/step only if it holds the corresponding name.
- NFR-4** *Attested execution.* World-effecting steps are confirmed only by signed oracle attestations; the coordinator never asserts physical completion itself.
- NFR-5** *Auditability.* Provenance, grounding, attestations, templates, and caveats are all on-channel and replayable.
- NFR-6** *Monotone learning.* Templates and caveats accumulate and feed future matching without mutating past records.

9 Worked example

Alice wants a catered dinner delivered at a venue on a date. As a Σ -bundle (record), her PF demands a **Venue**, a **Chef**, and a proof that the chef's cuisine is licensed for the venue — the last being a *dependent* field, which is what makes the types do real work.

```
// Alice's PF (schematic PFLam term). Two agent holes hVenue, hChef and a
// dependent field whose type mentions both fillers.
alicePF =
  Sig (v : Venue) .
  Sig (c : Chef) .
  LicensedFor (cuisine c) (kitchen v)           // dependent: depends on v and c
  with holes
    ? hVenue : Venue
    ? hChef  : Pi (v : Venue) . Chef           // chef may depend on the chosen venue

// 1. PUBLISH
publish!( alice, "dinner", alicePF )
//   → stores @[alice,"pf","dinner"], arms demands for hVenue and hChef

// 2. OFFERS (supply)
@[bob,   "supply"]!!( bobVenue ) // bobVenue  /= Venue
@[carol, "supply"]!!( carolChef ) // carolChef /= Pi (v:Venue).Chef

// 3. BROKER forwards type-matching candidates
broker!( bob,   alice, "dinner", "hVenue", Venue )
broker!( carol, alice, "dinner", "hChef",   Pi (v : Venue) . Chef )

// 4. MATCH + PLUG (happens behind Alice's guards)
//   hVenue ← bobVenue, hChef ← carolChef ; provenance recorded
//   each plug republishes; after both, holes("dinner") = [] → GROUND

// 5. RUN: dispatch obligations from provenance
//   @[bob,  "todo","dinner",0]!( "prepare & open venue" )
//   @[carol,"todo","dinner",1]!( "cater dinner" )

// 6a. SUCCESS: both attest "ok"
```

```

@[alice,"step","dinner",0]!( "ok" )    // bob done
@[alice,"step","dinner",1]!( "ok" )    // carol done
//    → abstract → @["mkt","templates"]!!( ["alice","dinner", template] )

// 6b. FAILURE (alternative): chef no-shows
@[alice,"step","dinner",1]!( "no-show" )
//    → fail → withdrawDemands ; then either
//    hide!( alice, "dinner" )           // quiet
//    or advertise!( alice, "dinner", "chef-no-show" ) // public caveat

```

The dependent field `LicensedFor` (cuisine `c`) (kitchen `v`) is the point: a venue/chef pair that type-checks is one for which a licensing proof *exists and is supplied*. The type system rejects incoherent groundings (an unlicensed pairing simply has no inhabitant for that field, so the bundle never type-checks, so the demand for that proof never fires), without anyone writing a bespoke rule.

10 Established vs. conjectural; open problems

We separate what the design *relies on and gets* from what it *hopes for*.

Established (given a normalizing fragment).

- *Acceptance is decidable.* Checking $q \Leftarrow T$ (hence $q = \text{ty}|$) is decidable when conversion is decidable; bidirectional checking against a *given* candidate needs no higher-order unification.
- *Plugging is total and structural;* the hole-set algebra $\text{holes}(p \triangleleft_h q) = (\text{holes}(p) \setminus \{h\}) \cup \text{holes}(q)$ holds definitionally.
- *The coordination layer is ordinary rho;* matching, brokering, joins/barriers, and persistent demands are exactly the embedded-rholang constructs MeTTaIL already provides.

Conjectural / delegated / out of scope for the calculus.

- *Filler search is undecidable in general.* Synthesizing a q with $q \models T$ is higher-order search; we deliberately delegate it to the market (§2.4). “Every PF eventually grounds” is a market *liveness* property, not a theorem; it depends on participation and incentives.
- *Full dependent types may not normalize / convert decidable* once one adds the features a real marketplace wants (large elimination, general recursion, effects). The supported fragment must be fixed deliberately; NFR-1 is a constraint on that choice, and it trades expressiveness for a decidable $=|$.
- *Templating as anti-unification.* “Reintroduce holes at filled positions” is clean for first-order positions; the principled general construction is least general generalization, which for typed/higher-order terms is subtle (no unique lgg in general). *abstract* is specified behaviourally here; its exact generalization order is open.
- *Execution is off-chain.* “All parties do their part in the external world” cannot be enforced by rho; NFR-4 reduces it to attestations from trusted oracles/timers. The trust model for those oracles, and the dispute resolution when attestations conflict, are out of scope here and should be designed next (this is also the natural seam for an economic/escrow layer).
- *Matching beyond checking.* If we later want the system to *propose* partial fillers (not just accept offered ones), we re-enter higher-order unification; a pragmatic middle path is Miller-pattern unification, decidable on the pattern fragment.

Suggested next decisions. (1) Fix the PFLam fragment and its conversion algorithm. (2) Specify `plan` and `abstract` precisely (ordering of obligations; generalization order). (3) Define the oracle/attestation trust model and where escrow attaches. (4) Decide whether brokers are privileged matchmakers, open auctions, or both, and what reputation signal (templates vs. caveats) they consume.

Glossary

PF	Plausible Fiction: a PFLam term with typed agent holes.
agent hole	A hole filled by another participant's PF; a market demand.
computation hole	A redex/elaboration metavariable; filled by reduction, never advertised.
grounded	No agent holes remain; the PF is runnable.
plug	$p \triangleleft_h q$: fill agent hole h in p with filler q (capture-avoiding).
 =	<i>models/satisfies</i> : $q = \text{ty}$ iff q checks at type ty ; the matching guard.
provenance	Map $\text{hole} \mapsto (\text{party}, \text{fillerId})$, used to dispatch obligations and to abstract.
template	A re-abstracted, reusable PF promoted from a success.
caveat	A publicly advertised failed pattern (anti-template).